

Swoosh: Efficient Lattice-Based Non-Interactive Key Exchange

Phillip Gajland^{1,2,3} & Miguel Quaresma¹

¹ Max Planck Institute for Security and Privacy

² Ruhr-University Bochum

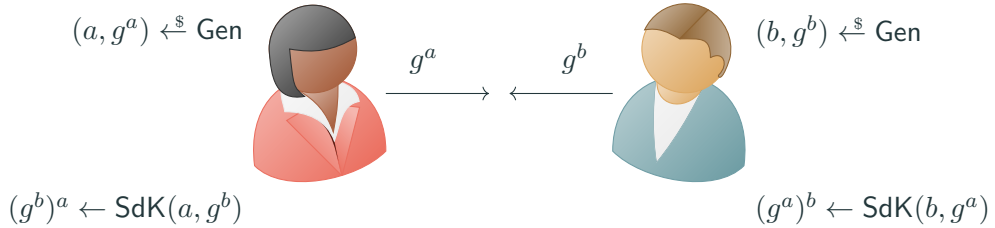
³ IBM Research Europe – Zurich

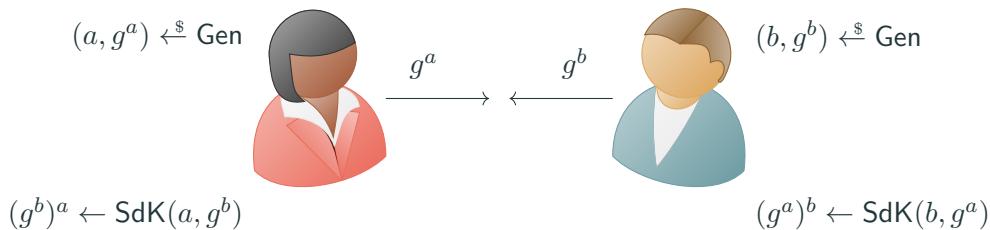
33rd USENIX Security Symposium, Philadelphia, USA

MAX PLANCK INSTITUTE
FOR SECURITY AND PRIVACY

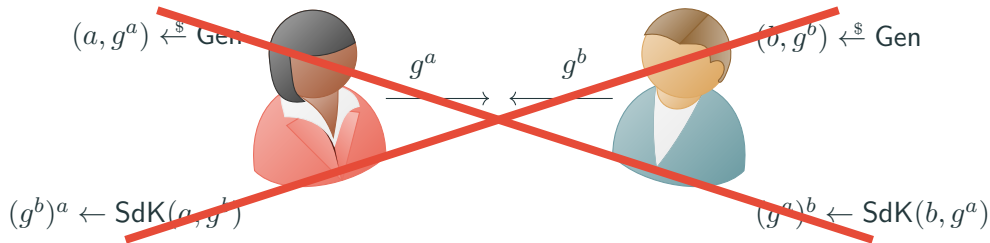


REPLACING DIFFIE-HELLMAN

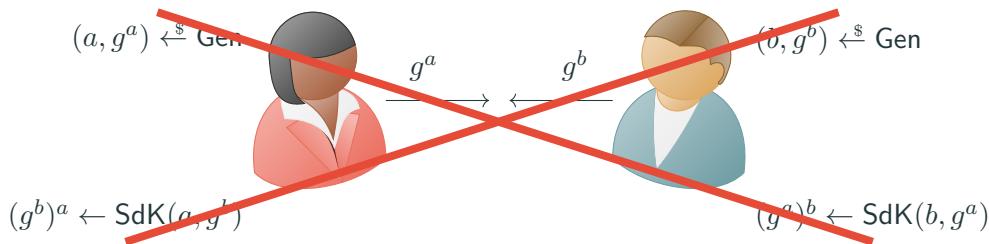




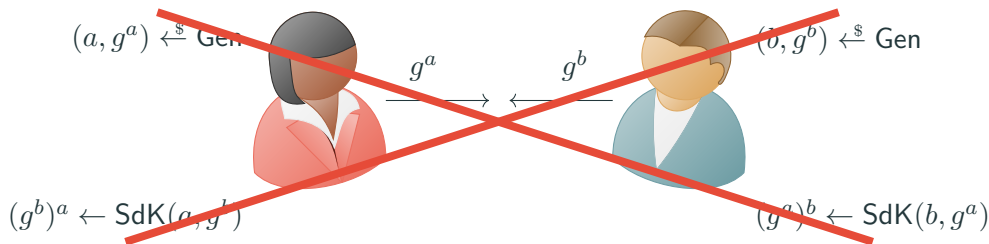
- Diffie-Hellman is widely deployed



- Diffie-Hellman is widely deployed
- Need a post-quantum replacement



- ▶ Diffie-Hellman is widely deployed
- ▶ Need a post-quantum replacement
- ▶ Folklore: “*Lattice-based non-interactive key exchange is impractical*”
 - ▶ Impossibility results [GKRS20]



- ▶ Diffie-Hellman is widely deployed
- ▶ Need a post-quantum replacement
- ▶ Folklore: “*Lattice-based non-interactive key exchange is impractical*”
 - ▶ Impossibility results [GKRS20]
- ▶ Our work: “*It’s not **that** bad*”

OUTLINE

- ▶ NIKE vs. KEM Applications
- ▶ Scheme: Passive-SWOOSH

- ▶ NIKE vs. KEM Applications
- ▶ Scheme: Passive-SWOOSH
- ▶ Parameter choices
- ▶ Implementation details
- ▶ Comparison

NON-INTERACTIVE KEY EXCHANGE (NIKE)



NON-INTERACTIVE KEY EXCHANGE (NIKE)

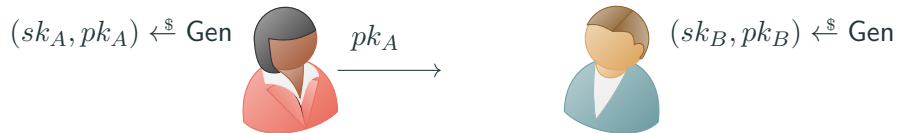
$(sk_A, pk_A) \xleftarrow{\$} \text{Gen}$



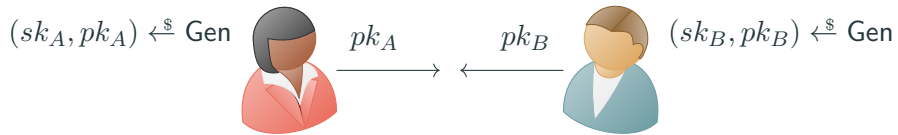
NON-INTERACTIVE KEY EXCHANGE (NIKE)



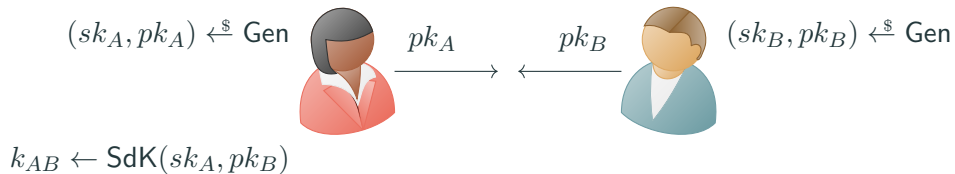
NON-INTERACTIVE KEY EXCHANGE (NIKE)



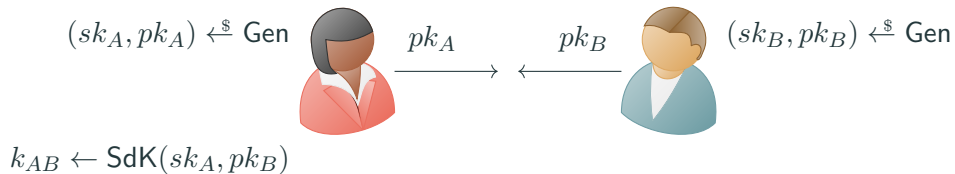
NON-INTERACTIVE KEY EXCHANGE (NIKE)



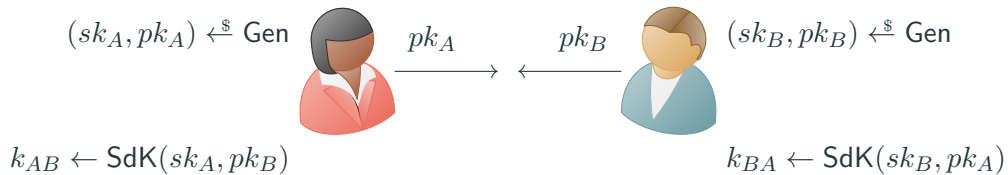
NON-INTERACTIVE KEY EXCHANGE (NIKE)



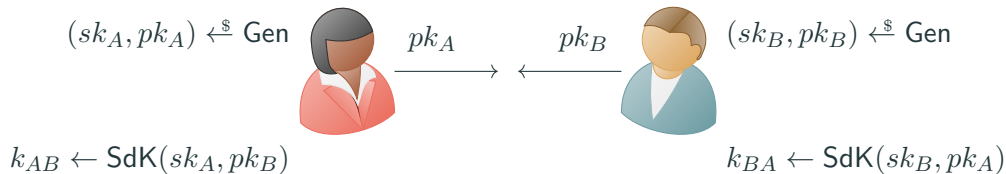
NON-INTERACTIVE KEY EXCHANGE (NIKE)



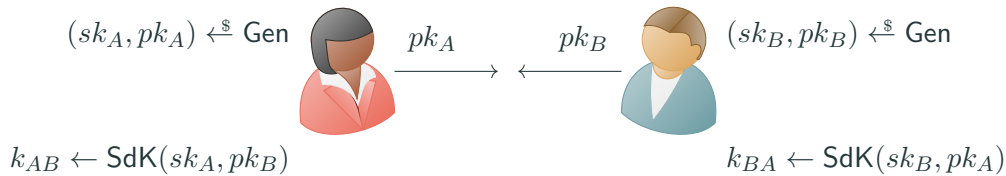
NON-INTERACTIVE KEY EXCHANGE (NIKE)



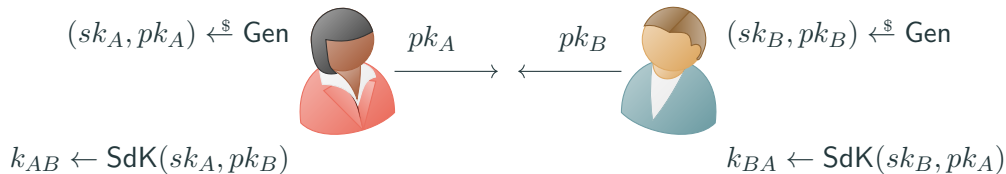
NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS



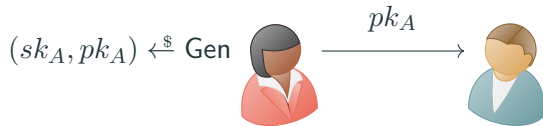
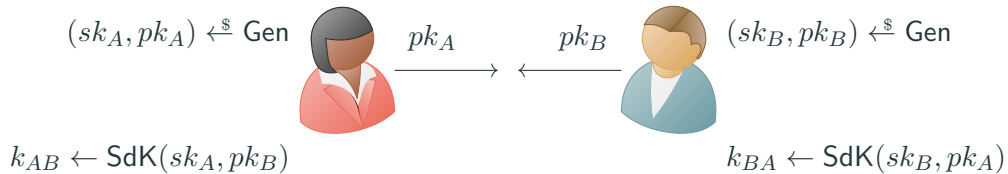
NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS



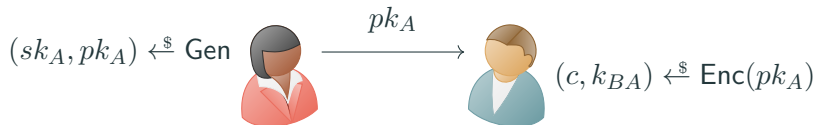
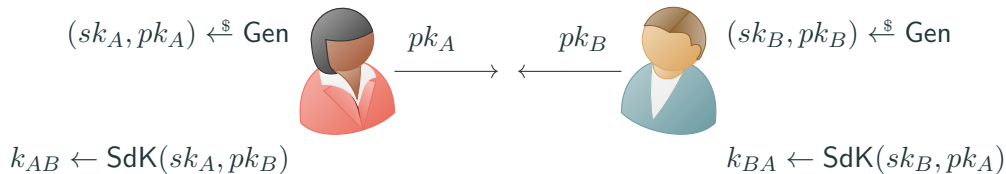
NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS



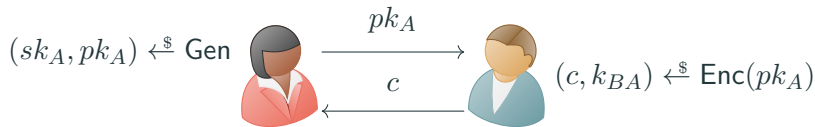
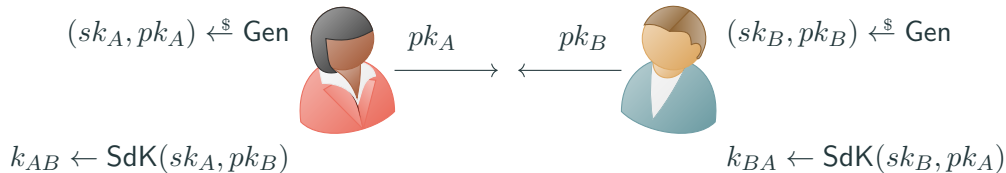
NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS



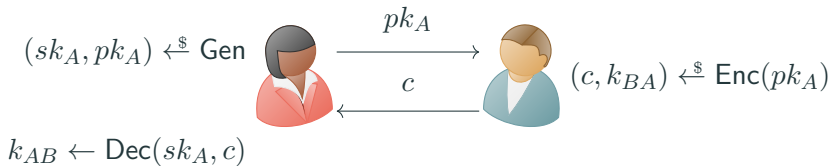
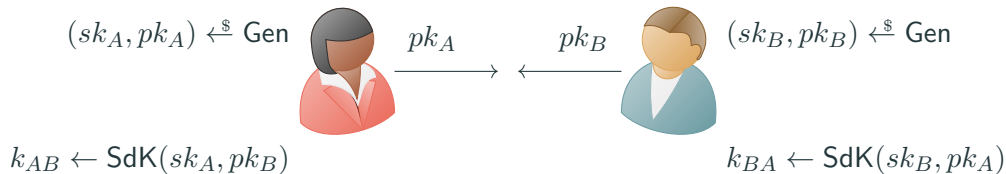
NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS

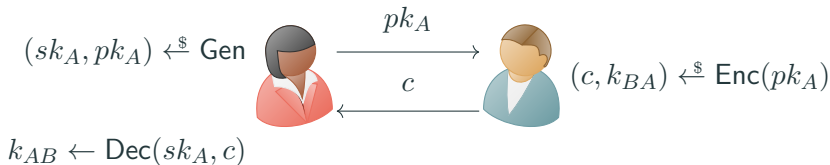
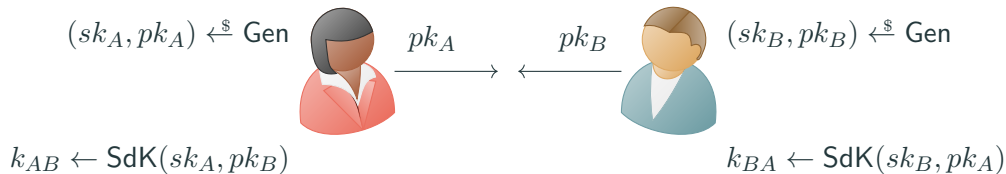


NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS

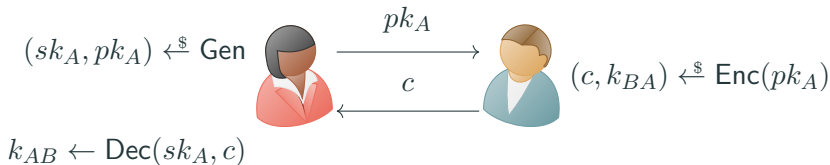
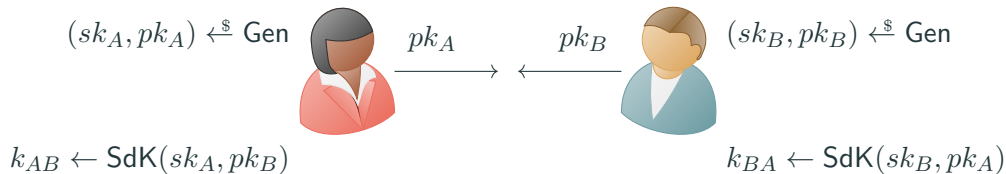


NON-INTERACTIVE KEY EXCHANGE VS. KEY-ENCAPSULATION MECHANISMS





► **Correctness:** $\delta := \Pr[k_{AB} \neq k_{BA}]$ should be small.



- **Correctness:** $\delta := \Pr[k_{AB} \neq k_{BA}]$ should be small.
- **Security:** k_{AB} and k_{BA} should look random.

- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?

- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?

- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?



- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?



- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?



- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?



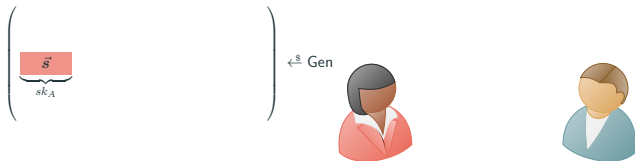
- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?



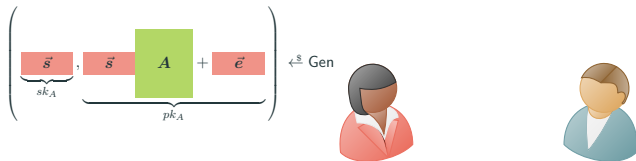
- ▶ Implicit Authentication using static Diffie-Hellman keys
 - ▶ OPTLS [KW16]
- ▶ Asynchronous key agreement
 - ▶ X3DH [MP16]
- ▶ Other
 - ▶ EDHOC [SMP24]
 - ▶ Group OSCORE [TSP⁺24]
 - ▶ More?



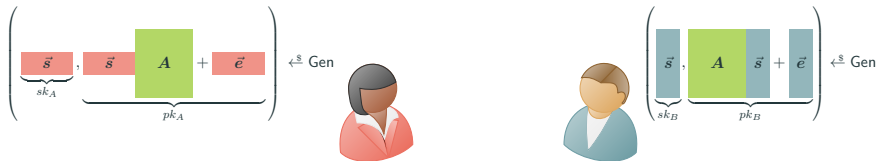
LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



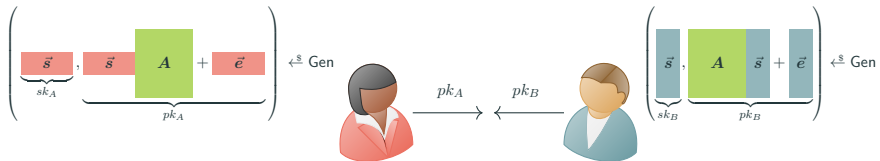
LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



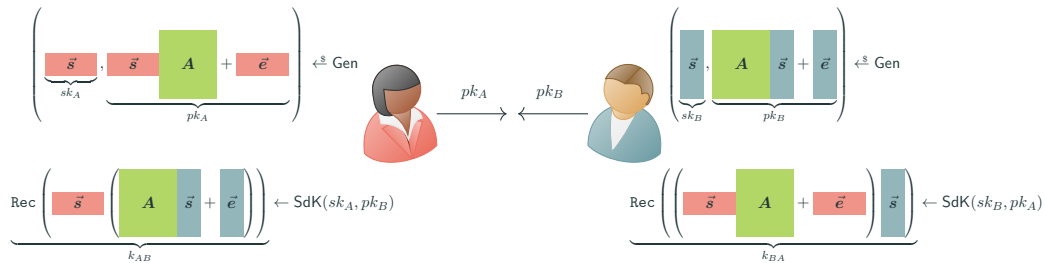
LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



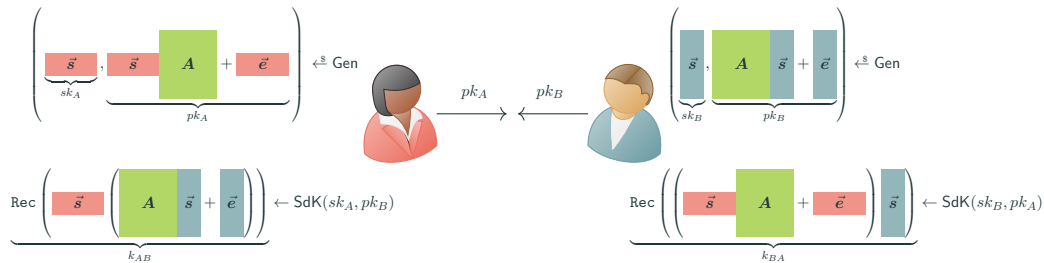
LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS

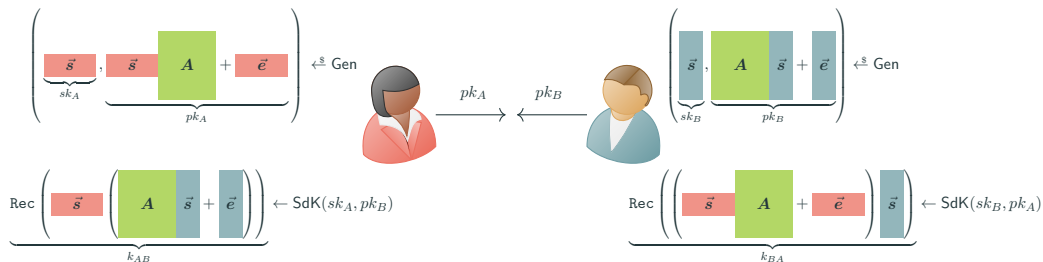


► Correctness:

$$\delta := \Pr \left[\text{Rec} \left(\vec{s}, \underbrace{A\vec{s} + \vec{e}}_{pk_B} + \vec{s}, \vec{e} \right) \neq \text{Rec} \left(\vec{s}, \underbrace{A\vec{s} + \vec{e}}_{pk_A} + \vec{e}, \vec{s} \right) \right] \leq \frac{4\beta^2 d^2 N}{q}.$$

► Security: k_{AB} and k_{BA} look random under the M-LWE assumption.

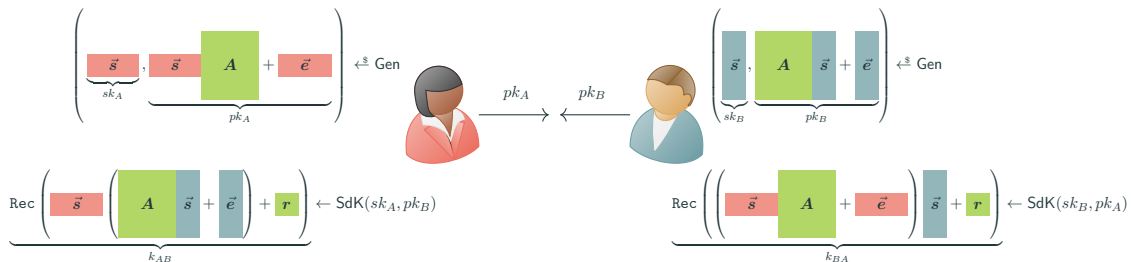
LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



► **Semi-Malicious Correctness:** (we add $\mathbf{r} := H(pk_A, pk_B)$)

► **Security:** k_{AB} and k_{BA} look random under the M-LWE assumption.

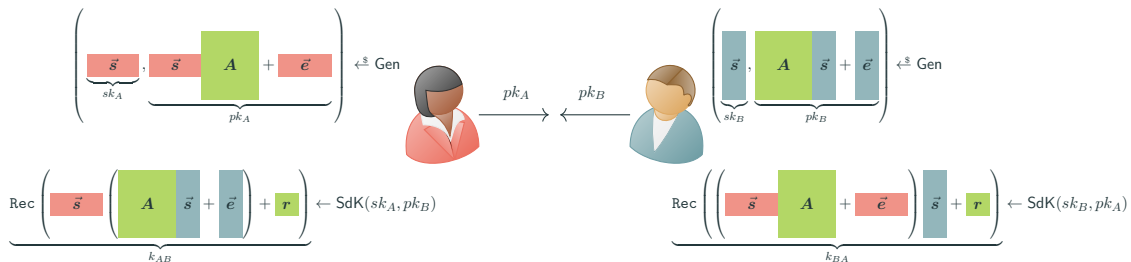
LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



► **Semi-Malicious Correctness:** (we add $\vec{r} := H(pk_A, pk_B)$)

► **Security:** k_{AB} and k_{BA} look random under the M-LWE assumption.

LATTICE-BASED NIKE HAS IMPERFECT CORRECTNESS



► **Semi-Malicious Correctness:** (we add $\mathbf{r} := \text{H}(pk_A, pk_B)$)

$$\delta := \Pr \left[\text{Rec} \left(\underbrace{\left(\text{red box } \vec{s} \right) \left(\text{green box } A \right) \left(\text{blue box } \vec{s} \right)}_{k_{AB}} + \underbrace{\left(\text{red box } \vec{s} \right) \left(\text{blue box } \vec{e} \right)}_{pk_B} + \underbrace{\left(\text{green box } r \right)}_{pk_A} \right) \neq \text{Rec} \left(\underbrace{\left(\text{red box } \vec{s} \right) \left(\text{green box } A \right) \left(\text{blue box } \vec{s} \right)}_{k_{BA}} + \underbrace{\left(\text{red box } \vec{e} \right) \left(\text{blue box } \vec{s} \right)}_{pk_A} + \underbrace{\left(\text{green box } r \right)}_{pk_B} \right) \right] \lesssim Q_H \cdot \frac{4\beta^2 d^2 N}{q}.$$

► **Security:** k_{AB} and k_{BA} look random under the M-LWE assumption.

- ▶ Diffie-Hellman has active security “almost for free” due to perfect correctness
- ▶ Passive-SWOOSH needs:
 - ▶ Semi-malicious correctness
 - ▶ Non-interactive zero-knowledge (NIZK) proof of knowledge
 - ▷ “I know the corresponding secret key of the public key”

- ▶ Diffie-Hellman has active security “almost for free” due to perfect correctness
- ▶ Passive-SWOOSH needs:
 - ▶ Semi-malicious correctness
 - ▶ Non-interactive zero-knowledge (NIZK) proof of knowledge
 - ▶ *“I know the corresponding secret key of the public key”*

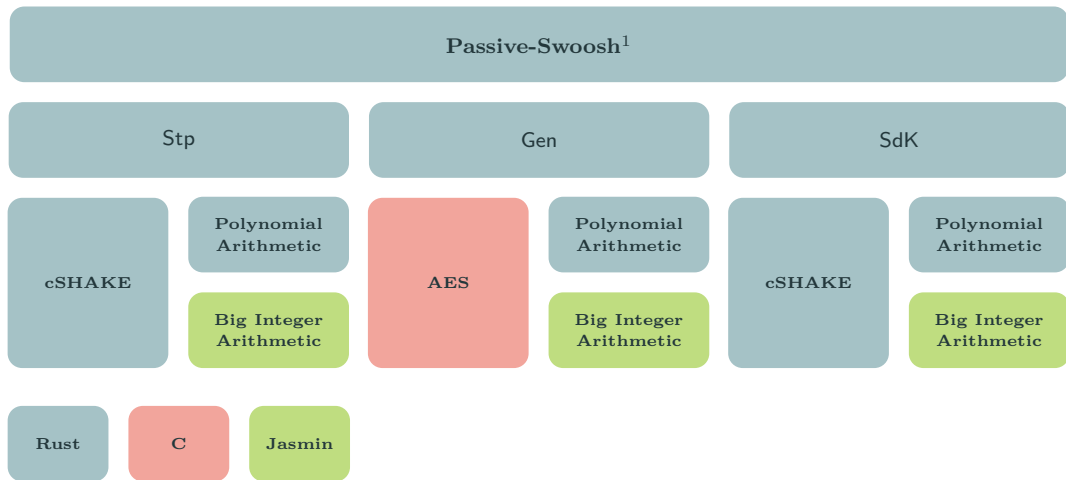
- ▶ Diffie-Hellman has active security “almost for free” due to perfect correctness
- ▶ Passive-SWOOSH needs:
 - ▶ Semi-malicious correctness
 - ▶ Non-interactive zero-knowledge (NIZK) proof of knowledge
 - ▶ *“I know the corresponding secret key of the public key”*

- ▶ Diffie-Hellman has active security “almost for free” due to perfect correctness
- ▶ Passive-SWOOSH needs:
 - ▶ Semi-malicious correctness
 - ▶ Non-interactive zero-knowledge (NIZK) proof of knowledge
 - ▶ *“I know the corresponding secret key of the public key”*

- ▶ Diffie-Hellman has active security “almost for free” due to perfect correctness
- ▶ Passive-SWOOSH needs:
 - ▶ Semi-malicious correctness
 - ▶ Non-interactive zero-knowledge (NIZK) proof of knowledge
 - ▶ *“I know the corresponding secret key of the public key”*

PARAMETERS AND IMPLEMENTATION

Parameter	Description	Value
β	upper bound on $\ \vec{s}\ _\infty = \ \vec{e}\ _\infty$	1
q	prime modulus	$2^{214} - 255$
d	dim of $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^d + 1)$	256
l	# factors $X^d + 1$ splits into mod q	128
N	height of the $\mathbf{A}$ matrix	32
n	lattice dimension	8192
χ	secret / noise distribution (ternary)	$p(-1) = 25\%$ $p(0) = 50\%$ $p(1) = 25\%$



¹NIZK not included

► Key Generation Gen

► Polynomial multiplication:

► Number Theoretic Transform (NTT) $\approx 10\%$

► Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation SdK

► Random offset computation

s_1 , e

$$r = H(pk_A, pk_B)$$

► Key Generation Gen

► Polynomial multiplication:

► Number Theoretic Transform (NTT) $\approx 10\%$

► Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation Sdk

► Random offset computation

s_1 , e

$$r = H(pk_A, pk_B)$$

► Key Generation Gen

► Polynomial multiplication:

► Number Theoretic Transform (NTT) $\approx 10\%$

► Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation Sdk

► Random offset computation

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

KeyGen

► Key Generation Gen

► Polynomial multiplication:

- Number Theoretic Transform (NTT) $\approx 10\%$
- Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation SdK

► Random offset computation



► Key Generation Gen

► Polynomial multiplication:

- Number Theoretic Transform (NTT) $\approx 10\%$

- Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation SdK

► Random offset computation

KeyGen

GenKey

GenKey

GenKey

GenKey

s_i , $\vec{e}$

GenKey

GenKey

$r = H(pk_A, pk_B)$

GenKey

GenKey

GenKey

GenKey

GenKey

► Key Generation Gen

► Polynomial multiplication:

- Number Theoretic Transform (NTT) $\approx 10\%$
- Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation Sdk

► Random offset computation

$\vec{s}_i$, $\vec{e}$

$r := H(pk_A, pk_B)$

► Key Generation Gen

► Polynomial multiplication:

- Number Theoretic Transform (NTT) $\approx 10\%$
- Big Integer Arithmetic

► Noise sampling

► Shared Key Derivation SdK

► Random offset computation

Gen

Gen

Gen

Gen

s_i , $\vec{e}$

Gen

$r := H(pk_A, pk_B)$

- ▶ Schoolbook multiplication is inefficient: $\mathcal{O}(d^2)$
- ▶ NTT is more efficient: $\mathcal{O}(d \log(d))$
 - ▶ In-place NTT reduces memory usage
 - ▶ NTT is part of the scheme (i.e. matrix A and public keys in NTT domain)

- ▶ Schoolbook multiplication is inefficient: $\mathcal{O}(d^2)$
- ▶ NTT is more efficient: $\mathcal{O}(d \log(d))$
 - ▶ In-place NTT reduces memory usage
 - ▶ NTT is part of the scheme (i.e. matrix $\mathbf{A}$ and public keys in NTT domain)

- ▶ Noise sampling 
 - ▶ Coefficients sampled from the output of a *Pseudo-Random Function* (PRF)
- ▶ Random offset computation 
 - ▶ Rejection sampling on output of an *eXtendable Output Function* (XOF)
- ▶ Instantiation of primitives dictates performance: AES256-CTR² & cSHAKE-256

²<https://bench.cr.yp.to/impl-stream/aes256ctr.html>

- ▶ Noise sampling , 
 - ▶ Coefficients sampled from the output of a *Pseudo-Random Function* (PRF)
- ▶ Random offset computation  $:= H(pk_A, pk_B)$
 - ▶ Rejection sampling on output of an *eXtendable Output Function* (XOF)
- ▶ Instantiation of primitives dictates performance: AES256-CTR² & cSHAKE-256

²<https://bench.cr.yp.to/impl-stream/aes256ctr.html>

- ▶ Noise sampling $\vec{s}_i$, $\vec{e}$
 - ▶ Coefficients sampled from the output of a *Pseudo-Random Function* (PRF)
- ▶ Random offset computation $\mathbf{r} := H(pk_A, pk_B)$
 - ▶ Rejection sampling on output of an *eXtendable Output Function* (XOF)
- ▶ Instantiation of primitives dictates performance: AES256-CTR² & cSHAKE-256

²<https://bench.cr.yp.to/impl-stream/aes256ctr.html>

COMPARISON OF SELECT POST-QUANTUM KEMS AND NIKES

Scheme (variant)	Assumption	PQ ³	NI ⁴	Size (bytes)		Cycles	
				<i>c</i>	<i>pk</i>	Gen	Enc + Dec or SdK
ECDH (X25519)	CDH	✗	✓	—	32	28 187	87 942
CRYSTALS-Kyber (Kyber-768)	M-LWE	✓	✗	1 088	1 184	200 302	539 108
Classic McEliece (mceliece348864)	Binary Goppa Codes	✓	✗	96	261 120	46 715 060	143 178
CTIDH (CTIDH-1024)	CSIDH	✓	✓	—	128	469 520 000	511 190 000
This work (Passive-SWOOSH)	M-LWE	✓	✓	—	221 184	146 920 890	10 612 666

³Post-quantum

⁴Non-interactive

COMPARISON OF SELECT POST-QUANTUM KEMS AND NIKES

Scheme (variant)	Assumption	PQ ³	NI ⁴	Size (bytes)		Cycles	
				<i>c</i>	<i>pk</i>	Gen	Enc + Dec or SdK
ECDH (X25519)	CDH	✗	✓	—	32	28 187	87 942
CRYSTALS-Kyber (Kyber-768)	M-LWE	✓	✗	1 088	1 184	200 302	539 108
Classic McEliece (mceliece348864)	Binary Goppa Codes	✓	✗	96	261 120	46 715 060	143 178
CTIDH (CTIDH-1024)	CSIDH	✓	✓	—	128	469 520 000	511 190 000
This work (Passive-SWOOSH)	M-LWE	✓	✓	—	221 184	146 920 890	10 612 666

- Diffie-Hellman is most efficient but not PQ secure
- PQC KEMs are faster than Passive-SWOOSH but require interaction
- PQ NIKEs: trade-off between key size and speed

³Post-quantum

⁴Non-interactive

SUMMARY

Contributions:

- ▶ M-LWE based NIKE, with strong correctness and proof in the QROM.
 - ▶ Generic transformation from passive to active security using NIZKs.
- ▶ Optimised implementation of Passive-SWOOSH, written in Rust and Jasmin.
 - ▶ Parameters achieving 120 bits of security against quantum adversaries.
 - ▶ Smaller public keys than Classic McEliece KEM and faster than CTIDH NIKE.



ia.cr/2023/271 — github.com/MQuaresma/pswoosh



youtu.be/Mo4g5z2Sea4



phillip.gajland@mpi-sp.org, rub.de

miguel.quaresma@mpi-sp.org



[p4i11ip](#), [miguel__mq](#)

Contributions:

- ▶ M-LWE based NIKE, with strong correctness and proof in the QROM.
 - ▶ Generic transformation from passive to active security using NIZKs.
- ▶ Optimised implementation of Passive-SWOOSH, written in Rust and Jasmin.
 - ▶ Parameters achieving 120 bits of security against quantum adversaries.
 - ▶ Smaller public keys than Classic McEliece KEM and faster than CTIDH NIKE.



ia.cr/2023/271 — github.com/MQuaresma/pswoosh



youtu.be/Mo4g5z2Sea4



phillip.gajland@mpi-sp.org, rub.de

miguel.quaresma@mpi-sp.org



[p4i11ip](#), [miguel__mq](#)

Contributions:

- ▶ M-LWE based NIKE, with strong correctness and proof in the QROM.
 - ▶ Generic transformation from passive to active security using NIZKs.
- ▶ Optimised implementation of Passive-SWOOSH, written in Rust and Jasmin.
 - ▶ Parameters achieving 120 bits of security against quantum adversaries.
 - ▶ Smaller public keys than Classic McEliece KEM and faster than CTIDH NIKE.



ia.cr/2023/271 — github.com/MQuaresma/pswoosh



youtu.be/Mo4g5z2Sea4



phillip.gajland@mpi-sp.org, rub.de

miguel.quaresma@mpi-sp.org



[p4i11ip](#), [miguel__mq](#)

Contributions:

- ▶ M-LWE based NIKE, with strong correctness and proof in the QROM.
 - ▶ Generic transformation from passive to active security using NIZKs.
- ▶ Optimised implementation of Passive-SWOOSH, written in Rust and Jasmin.
 - ▶ Parameters achieving 120 bits of security against quantum adversaries.
 - ▶ Smaller public keys than Classic McEliece KEM and faster than CTIDH NIKE.



ia.cr/2023/271 — github.com/MQuaresma/pswoosh



youtu.be/Mo4g5z2Sea4



phillip.gajland@mpi-sp.org, miguel.quaresma@mpi-sp.org



[p4i11ip](#), [miguel__mq](#)

Contributions:

- ▶ M-LWE based NIKE, with strong correctness and proof in the QROM.
 - ▶ Generic transformation from passive to active security using NIZKs.
- ▶ Optimised implementation of Passive-SWOOSH, written in Rust and Jasmin.
 - ▶ Parameters achieving 120 bits of security against quantum adversaries.
 - ▶ Smaller public keys than Classic McEliece KEM and faster than CTIDH NIKE.



ia.cr/2023/271 — github.com/MQuaresma/pswoosh



youtu.be/Mo4g5z2Sea4



phillip.gajland@mpi-sp.org, miguel.quaresma@mpi-sp.org

miguel.quaresma@mpi-sp.org



[p4i11ip](https://twitter.com/p4i11ip), [miguel__mq](https://twitter.com/miguel__mq)

Contributions:

- ▶ M-LWE based NIKE, with strong correctness and proof in the QROM.
 - ▶ Generic transformation from passive to active security using NIZKs.
- ▶ Optimised implementation of Passive-SWOOSH, written in Rust and Jasmin.
 - ▶ Parameters achieving 120 bits of security against quantum adversaries.
 - ▶ Smaller public keys than Classic McEliece KEM and faster than CTIDH NIKE.



ia.cr/2023/271 — github.com/MQuaresma/pswoosh



youtu.be/Mo4g5z2Sea4



phillip.gajland@mpi-sp.org, rub.de

miguel.quaresma@mpi-sp.org



[p4i11ip](#), [miguel__mq](#)



- [ABC⁺22] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- [BBC⁺21] Gustavo Banegas, Daniel J. Bernstein, Fabio Campos, Tung Chou, Tanja Lange, Michael Meyer, Benjamin Smith, and Jana Sotáková. CTIDH: faster constant-time CSIDH. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):351–387, 2021. <https://tches.iacr.org/index.php/TCHES/article/view/9069>.
- [Ber06] Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006: 9th International Conference on Theory and Practice of Public Key Cryptography*, volume 3958 of *Lecture Notes in Computer Science*, pages 207–228, New York, NY, USA, April 24–26, 2006. Springer, Heidelberg, Germany.
- [GKRS20] Siyao Guo, Pritish Kamath, Alon Rosen, and Katerina Sotiraki. Limits on the efficiency of (ring) LWE based non-interactive key exchange. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 12110 of *Lecture Notes in Computer Science*, pages 374–395, Edinburgh, UK, May 4–7, 2020. Springer, Heidelberg, Germany.
- [KW16] Hugo Krawczyk and Hoeteck Wee. The OPTLS protocol and TLS 1.3. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 81–96, Saarbruecken, Germany, March 2016. IEEE.
- [MP16] Moxie Marlinspike and Trevor Perrin. The X3DH key agreement protocol (revision 1). Part of the Signal Protocol Documentation, 2016. <https://signal.org/docs/specifications/x3dh/x3dh.pdf>.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.

- [SMP24] Göran Selander, John Preuss Mattsson, and Francesca Palombini. Ephemeral Diffie-Hellman Over COSE (EDHOC). Internet-Draft draft-ietf-lake-edhoc-23, Internet Engineering Task Force, January 2024. Work in Progress.
- [TSP⁺24] Marco Tiloca, Göran Selander, Francesca Palombini, John Preuss Mattsson, and Rikard Höglund. Group Object Security for Constrained RESTful Environments (Group OSCORE). Internet-Draft draft-ietf-core-oscore-groupcomm-21, Internet Engineering Task Force, March 2024. Work in Progress.